Overview of Kalman Filter Theory and Navigation Applications

Day 1

Michael L. Carroll

June 7, 2007

©2007 by Michael L. Carroll

Course Overview

- Topic Overview
- Course Structure
- Detailed Topic List
- Instructor Background

Course Overview: Topic Overview

- Kalman Filtering
  - General Theory Overview w/o proofs
  - Practical Aspects of Application
  - Matlab and Simulink Examples
- Applications to GPS Control Segment

Course Overview: Course Structure

- 10 segments (based on 5-day engineering short course)
- 1 hour per segment
- Informational Course
  - Concept
  - Example
  - Exercise (optional)

Course Overview: Detailed Topic List

- Segments 1-2: The Basic Kalman Equations, Part I – Heuristic Overview

- Segments 3-4: The Basic Kalman Equations, Part II – State Dynamics, Probability, Statistics and Random Processes

- Segment 5-6: General Code Implementation Issues and Applications to Navigation

- Segments 7-10: Details of the GPS Control Segment Kalman Filter

Course Overview: Instructor Background

- Michael Carroll
- Masters Degree in Mathematics
- Software Integrator on GPS SE&I Contract

Course Overview: Instructor Background
Formerly ...

- Software Architect on C-130 Avionics Modernization Program at Boeing in Long Beach, CA

- Worked with Calibration and Alignment filters for ballistic missile inertial measurement units (IMU)

- Led avionics software development efforts on various USAF Special Operations Command programs at Lockheed Aircraft Service Company (LASC) in Ontario, CA

Course Overview: Instructor Background
Formerly ...

- GPS/INS Navigation Analyst at BEI Systron Donner

- Have taught formal Kalman filtering courses several times

Kalman Filter Overview

- Optimal state estimator of uncertain dynamic system using noisy measurements
- Essentially recursive least squares
- Rudolf E. Kalman
  Seminal paper in 1960: "A New Approach to Linear Filtering and Prediction Problems"

`http://www.cs.unc.edu/~welch/kalman/kalmanPaper.html`

Kalman Filter Overview

- Actually preceded by work at RAND Corp by Peter Swerling in 1950s
- Least squares goes back to Gauss, Lagrange late 1700s and early 1800s
- Rigorous mathematical foundation built on work of mathematician David Hilbert
- Kalman worked in the time domain using state space techniques

Kalman Filter Overview

- Used in early 1960s by Schmidt on Apollo program
- Kalman filters are now ubiquitous, especially in navigation

Kalman Filter Overview

- GPS uses:
  - All three segments have Kalman filters
  - CS: ECEST = Ephemeris and Clock Estimator (legacy term)
  - Estimates inertial positions and velocities of SVs, solar pressure parameters, SV and MS clock parameters, and troposphere state at each MS
  - In UE receivers, satellite nav software, and offline computations

– Used in aircraft Operational Flight Programs (OFPs) and Inertial Navigation Systems (INSs) to integrate GPS and other external navaids to optimally estimate INS error states.

– Paradigm shift in navigation:
Was: "INS/GPS"
Is: "GPS/INS".

- Kalman filtering, as a branch of applied mathematics, sits at the confluence of many mathematical subjects:
  differential equations, measure theory, probability, statistics, linear algebra, functional analysis (Hilbert space).

- But its essentials are very intuitive and can be understood with very little math background.

Some Notational Conventions

- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)

- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$

- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)

- Corresponding matrix components (elements):
  $p_{ij}(k)$ or $p_{ij}(t)$

Day 1: The Basic Kalman Equations, Part I
Segments

- Understanding the Equations: Heuristic Overview
- Taking the Equations Apart
- Differential Equations, Difference Equations and Dynamic Systems
- State Space Concepts

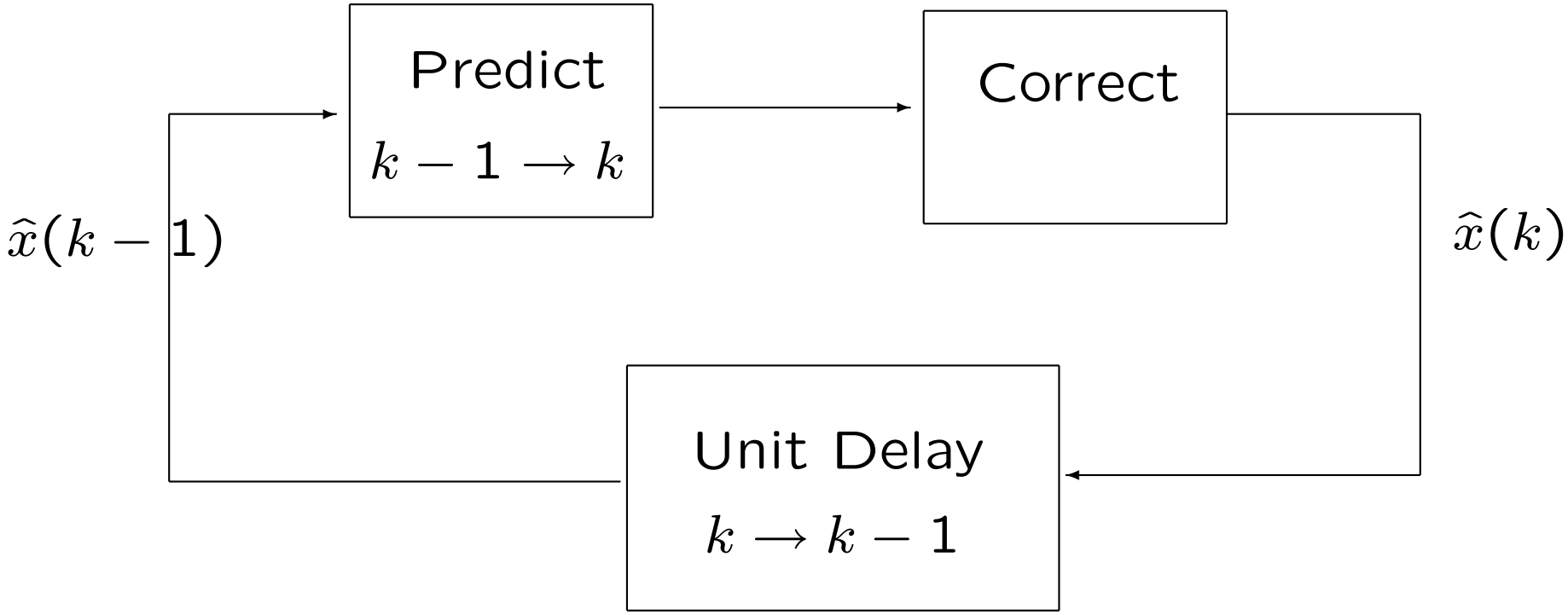
Day 1, Segment 1
Understanding the Equations: Heuristic Overview
Topics

- Recursive Predictor-Corrector Algorithms
- Running Averages
- Extrapolation (Prediction)
- Gain Computation
- Update

Recursive Predictor-Corrector Algorithm

- Prediction:
  - Kalman filter **extrapolates** the solution from the previous iteration to the present time
- Correction:
  - Kalman filter uses measurement information to **update** or **correct** the prediction
- And the cycle begins anew!

Kalman Filter Predictor-Corrector Model



Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, new quantities are computed
- Recursive
  - The new quantities are the same as the old quantities
  - New quantities become initial values for next cycle

Simple Example: Running Averages

- Suppose we want to estimate some quantity like the resistance of a single resistor.

- Suppose we have a voltmeter with which to make measurements.

- Thus we have two distinct quantities: the "true" value x of the resistor and the measurement z given by the voltmeter.

- We are going to make an indefinite number of measurements: $z(1), z(2), \ldots$

Simple Example: Running Averages

- The **state** that we are trying to estimate is the resistance of the resistor.

- We don't know what the state is. Otherwise, we wouldn't need to take measurements.

- Our **state dynamics** or **process model** is very simple: The state is 1-dimensional and does not change with time:

$$x(k+1) = x(k)$$

Simple Example: Running Averages

- What is the best way to estimate the state of our system?
- In the absence of any other information, we should probably just average the measurements.
- We assume that the averages are unbiased, i.e., the measurement errors have zero mean.

Simple Example: Running Averages

- Simple running average is like a Kalman filter with trivial dynamics ($x(k+1) = x(k)$)

Given an infinite sequence of random measurements $z(1), z(2), \ldots$, compute at each stage the best estimate of the true value of the underlying quantity: At the $n+1^{\text{st}}$ step, your best guess would be the average of all previous values:

$$(1) \qquad \widehat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,]$$

Simple Example: Running Averages

- Q: Why is the average of all previous measurements the "best" estimate?

- A: The dynamics tell us that the quantity to be estimated is not changing, and the only variability stems from the measurements, i.e., the measurement noise.

- Hence, we should try to eliminate the measurement variability by averaging the measruements.

Recursive Running Average

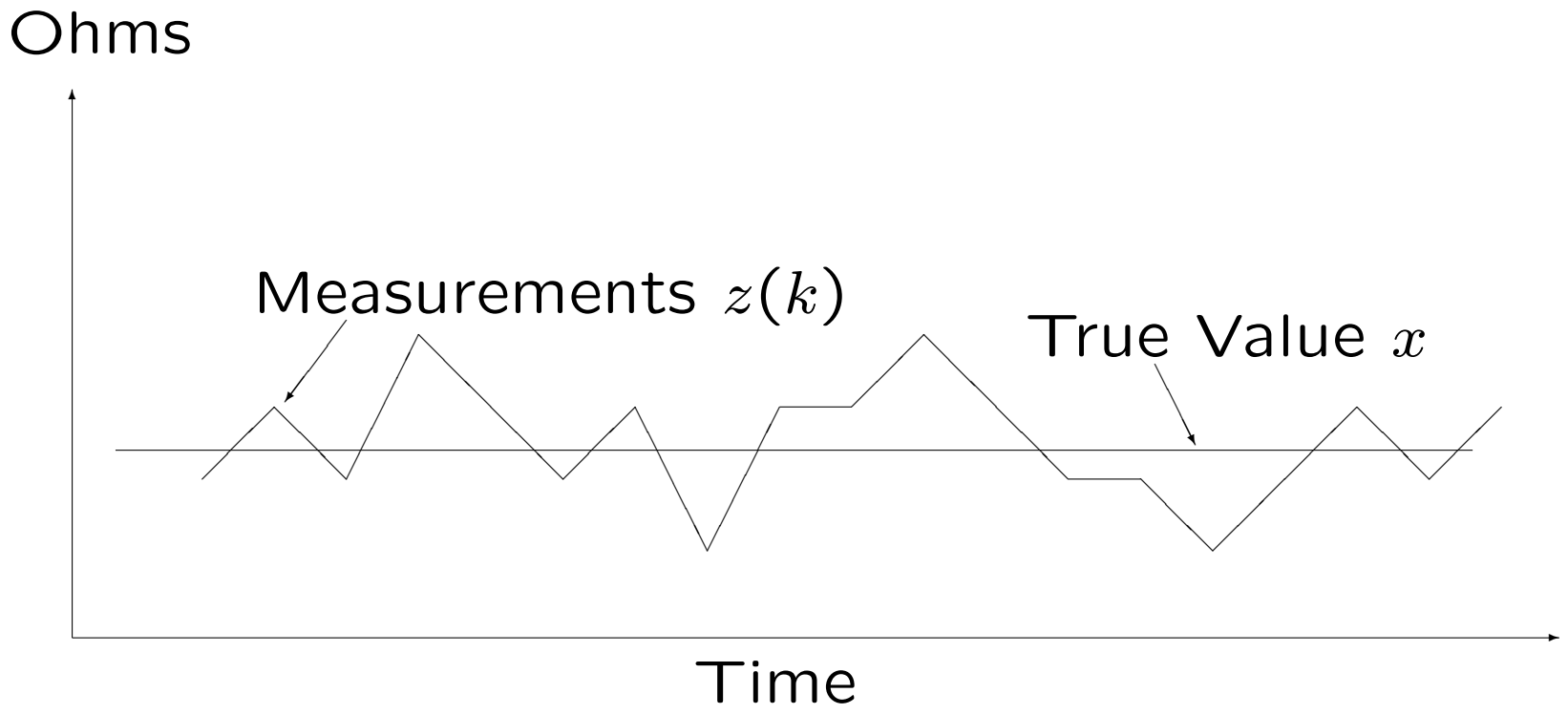
- Therefore, as much as possible, we want to rid ourselves of the noise by averaging.

- Assume that noise is zero-mean, i.e., its average tends to zero over the long run.

Recursive Running Average

- If we had a bias, we wouldn't call that noise. That would be a systematic error on the part of our voltmeter. We'd have to calibrate it out.

- Noise can't be calibrated out; we have to average it out over time.


Ohms
Measurements $z(k)$
True Value x
Time

Recursive Running Average (Eq.(1))

$$\widehat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,]$$

- Now suppose we are lazy and don't want to recalculate the complete sum with each new measurement.

- We just want to use the previously calculated value and add a small adjustment:

Recursive Running Average

$$\widehat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{separate last term from sum}$$

$$= \frac{1}{k}\cdot\frac{k-1}{k-1}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{multiply by 1}$$

$$= \frac{k-1}{k}\cdot\frac{1}{k-1}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{rearrange}$$

$$= \frac{k-1}{k}\widehat{x}(k) + \frac{1}{k}z(k) \qquad \text{by def. Eq. (1) again}$$

$$= \widehat{x}(k) - \frac{1}{k}\widehat{x}(k) + \frac{1}{k}z(k) \qquad \text{distribute and simplify}$$

$$= \widehat{x}(k) + \frac{1}{k}\left[\, z(k) - \widehat{x}(k) \,\right]. \qquad \text{rearrange and factor out } 1/n$$

Recursive Running Average

- We now have a recursive algorithm for computing a running average
  - Recipe: Take new measurement, subtract off old average and weight the resulting residual by a gain of $1/k$.
  - Add this weighted residual to the old average and, voilà, you have the new average!

Recursive Running Average

- Note that you do not have to remember all the previous values or averages, only the last average.

- Note also that as k goes to ∞, the gain $1/k$ goes to 0. (The filter is going to sleep!)

$$\widehat{x}(k+1) = \widehat{x}(k) + \frac{1}{k}\left[z(k) - \widehat{x}(k)\right] \tag{2}$$

Recursive Running Average

- The residual $z(k) - \widehat{x}(k)$ is the difference between the actual new measurement and what we expected or predicted the new measurement to be.

- Why is $\widehat{x}(k)$ our predicted measurement?

- Answer: In the absence of new information, the previous estimate is the only thing we have to go on!

Extrapolation (Prediction)

- In the running average example, the dynamics were trivial, i.e., $x(k+1) = x(k)$

- This means that the state never really changes. It's always equal to its previous value.

- We just don't know what the exact value is.

- Example: Measuring the same resistor over and over again with a voltmeter to estimate the resistor's resistance value.

Extrapolation (Prediction)

- In more general Kalman filtering, we complicate the situation by adding two more elements:
  - State dynamics with time dependence and process noise:

$$x(k+1) = g\left[k, x(k), w(k)\right]$$

  - State dynammics with additive process noise:

$$x(k+1) = f\left[k, x(k)\right] + w(k)$$

- The form of the function f can be simple or complicated. The independent variable k indicates possible explicit time dependence.

- The process noise is assumed to be Gaussian white noise (to be defined later).

Extrapolation (Prediction)

- General Kalman filtering further complicates things by allowing the state variable to be vector-valued.

- This turns the function f into a vector-valued function, often a matrix (possibly time-dependent)

- The process noise must therefore also become a vector-valued variable.

Extrapolation (Prediction)

- Note that the presence of a random forcing function like white noise turns the dynamical problem into a stochastic one.

- If the white noise were zero, then the problem would be deterministic.

- We'll see what this means when we take a look at random variables and random processes.

Extrapolation (Prediction)

- Note that we did not mention a control vector u which would also be part of the forcing function.

- We are generally going to ignore the presence of control.

- The control vector is usually considered to be deterministic although in reality you can't control things perfectly! (E.g., elevator deflection angle might be a control quantity and its precise value may not be known!)

Measurement Update (Correction)

- In the running average example, our update implicitly assumed that our measurement was corrupted by white noise.

- A model for this would be: $z(k) = x(k) + v(k)$, where $v(k)$ is a Gaussian white measurement noise sequence.
  - We'll explore what we mean by "Gaussian" later in the course.

Measurement Update (Correction)

- Because we assume that our measurement noise was white Gaussian and zero mean, our best guess was simply the mean value of the measurements.

- Remember the resistor example: our voltmeter is not perfect. Each measurement will differ slightly from the previous one, even though we are measuring the same resistor each time.

Measurement Update (Correction)

- General Kalman filtering complicates the measurement model by adding two new elements:
  - The measurement may have a more complicated relationship to the underlying system state:

    $$z(k) = h\left[k, x(k)\right] + v(k),$$

    where h could be a non-linear function with possible explicit time dependence.
  - Just as the state x is allowed to be a vector variable, so the measurement z is also allowed to be a vector variable.

Kalman Gain Computation

- Even through we've discussed the measurement update or observation processing step, there is an important step just before that one.

- Before we can incorporate the information contained in both the old estimate and the new measurement, we have to first calculate the Kalman gain K_k at the kth step.

- $K_k = \frac{1}{k}$ in our running average example.

- If we have k pieces of information *of equal quality*, it seems intuitive that we weight each piece by $1/k$.

Kalman Gain Computation

- What we are weighting is the measurement residual, also called the **innovation**: $z(k) - \widehat{x}(k)$.

- The innovation contains all the new information, i.e., the new measurement, minus what we thought we already knew, i.e., the previous estimate.

- The gain computation occurs after the extrapolation but before updating our estimate with measurement information.

Day1, Segment 1 - Exercises

1. Let's re-familiarize ourselves with vectors and matrices. Addition:

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} x_1 + y_1 \\ x_2 + y_2 \\ \vdots \\ x_n + y_n \end{bmatrix}$$

For vectors in three space, this would be:

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} x_1 + y_1 \\ x_2 + y_2 \\ x_3 + y_3 \end{bmatrix}$$

Compute:

$$\begin{bmatrix} -1 \\ 3 \\ 0 \end{bmatrix} + \begin{bmatrix} 7 \\ -1 \\ 5 \end{bmatrix} =$$

$$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix} =$$

$$\begin{bmatrix} 0.75 \\ -3 \\ 100 \end{bmatrix} + \begin{bmatrix} 0.75 \\ 0 \\ -1000 \end{bmatrix} =$$

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} =$$

$$\begin{bmatrix} 1.1 \\ -3.2 \\ 9.7 \end{bmatrix} + \begin{bmatrix} -0.8 \\ 3.2 \\ 10 \end{bmatrix} =$$

2. Multiplying a row vector times a column vector:

$$\begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = a_1 b_1 + a_2 b_2 + a_3 b_3$$

Now you try it:

$$\begin{bmatrix} 0 & 1 & 3 \end{bmatrix} \begin{bmatrix} 4 \\ 3 \\ 0 \end{bmatrix} =$$

$$\begin{bmatrix} -1 & 3 & -2 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \\ 2 \end{bmatrix} =$$

$$\begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix} =$$

$$\begin{bmatrix} 2x & .5x & -7x \end{bmatrix} \begin{bmatrix} -10 \\ 4 \\ -1 \end{bmatrix} =$$

3. Multiplying a matrix times a column vector:

$$\begin{bmatrix} 1 & 0 & 1 \\ 2 & 7 & 0 \\ 3 & 2 & 5 \end{bmatrix} \begin{bmatrix} 7 \\ -1 \\ 5 \end{bmatrix} =$$

Hint: Multiply the first row of the matrix times the column vector to get the first element of the new column vector. Then multiply by the second row of the matrix, and so on.

$$\begin{bmatrix} 2 & 1 & 0 \\ 0 & 1 & 1 \\ 2 & 2 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ -3 \\ 1 \end{bmatrix} =$$

$$\begin{bmatrix} 3 & 1 & 1 \\ -1 & 1 & -1 \\ 0 & 2 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ -3 \\ 1 \end{bmatrix} =$$

4. What happens if you multiply a column on the left by a row vector on the right?

5. What is the transpose of the following matrix?

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

6. Given the following scalar measurements of some constant scalar process, calculate the average value at each stage k by two different methods: 1) by averaging all past values and 2) by using the recursive algorithm in Eq. (2):

1. $z_1 = 0.76$

2. $z_2 = 0.67$

3. $z_3 = 0.79$

4. $z_4 = 0.81$

5. $z_5 = 0.78$

6. $z_6 = 0.79$

7. $z_7 = 0.81$

8. $z_8 = 0.82$

9. $z_9 = 0.77$

7. You are traveling in a car at 45 mph in a 35 mph speed zone. Your friendly neighborhood policeman is watching you with his radar gun. He is measuring your speed, of course, but needs to report as well your position as a function of time. Write down a dynamics and measurement model for this problem in which the state variable is your positional displacement from some arbitrary initial position. Assume that there is no process noise in your dynamics and that your velocity is not changing. Assume the policeman makes a measurement with his radar gun every 2 seconds.
Hint: You have two variables of interest! 8. How is this problem different from the resistor problem?

Day 1, Segment 2
Taking the Equations Apart
Topics

- Understanding the Notation
- Examples
- Exercises

Understanding the Notation: Kalman Filter Problem Summary

- Given two models:

  1. State Dynamics Model (sometimes called System or Process Model)

  2. Measurement Model

- Find the optimal linear gain K_k with which to weight the measurement residual or innovation in order to compute the optimal estimate of the state (we'll discuss later what we mean by "optimal")

Understanding the Notation: Kalman Filter Problem Summary (General, Non-Linear, Discrete Formulation)

- Given: State Dynamics Model: $x(k) = f[x(k-1)] + w(k)$
  - x=state,f=dynamics function,w=process noise
- Given: Measurement Model: $z(k) = h[x(k)] + v(k)$
  - z=measurement, h=measurement function, v=measurement noise
- Find optimal gain K_k that allows us to update our state estimate in the following way:

$$\widehat{x}(k) = f[\widehat{x}(k-1)] + K_k\ [\ z(k) - h(f[\widehat{x}(k-1)])\ ]$$

Understanding the Notation: Kalman Filter Problem Summary

- Note that we seek the gain in the form of a multiplier (i.e., a gain matrix)

- This is not the most general form possible. The gain could be a (possibly, non-linear) function of the innovation, perhaps something like $g_k(z(k), \widehat{x}(k))$. But we won't pursue that.

Understanding the Notation: Kalman Filter Problem Summary (Linear Discrete Formulation)

- Given: State Dynamics Model: $x(k) = \Phi_{k-1}x(k-1) + w(k)$
  - $x(k)$=state,Φ_{k-1}=state transition matrix, $w(k)$=process noise
- Given: Measurement Model: $z(k) = H_k x(k) + v(k)$
  - $z(k)$=measurement,H_k=measurement matrix, $v(k)$=measuremen noise

- Find: State estimate in the form

$$\widehat{x}(k) = \Phi_{k-1}\widehat{x}(k-1) + K_k[z(k) - H_k\Phi_{k-1}\widehat{x}(k-1)]$$

Understanding the Notation: Kalman Filter Problem Summary
Slight Notational Change

- The term $\Phi_{k-1}\widehat{x}(k-1)$ is the extrapolated state estimate prior to making the measurement update or correction.

- To distinguish prior estimate from updated estimate, we use the following notation:

  $\widehat{x}^{-}(k) = \Phi_{k-1}\widehat{x}^{+}(k-1) =$ predicted estimate prior to update

  $$\widehat{x}^{+}(k) = \text{updated state estimate}$$

Understanding the Notation: Kalman Filter Problem Summary

- Thus, the solution we seek looks like this:

(3) $$\widehat{x}^{+}(k) = \widehat{x}^{-}(k) + K_k[z(k) - H_k\widehat{x}^{-}(k)]$$

Understanding the Notation: Kalman Filter Problem Summary

- We've formulated the discrete problem
- We've indicated the form of the solution: the recursive estimating algorithm with the Kalman gain
- But we haven't yet presented the general solution. Only in the case of a simple, constant, scalar system.
- When we arrive at the general solution, we'll get an added benefit: While trying to come up with $\widehat{x}^{+}(k)$ and K_k, we'll also end up with a recursive estimate of our estimation uncertainty! This is the so-called error covariance matrix.

Understanding Kalman Filter Notation
State Dynamics

- State Dynamics Model: $x(k) = f[x(k-1)] + w(k)$
- $x(k)$ is the **state vector** with n elements
- $x(k) = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}_k = \begin{bmatrix} x_1(k) \\ x_2(k) \\ \vdots \\ x_n(k) \end{bmatrix}$

Understanding Kalman Filter Notation
State Dynamics

- Discrete State Dynamics
- General, nonlinear dynamics function f too difficult!
- Restrict to matrices: $x(k) = \Phi_k x(k-1) + w(k)$
- Φ_{k-1} is the state transition matrix. It is $n \times n$ in size.

Understanding Kalman Filter Notation
State Dynamics

- State Transition Matrix: $\Phi_{k-1} = \Phi(k, k-1)$
- Extrapolates state vector from time step $k-1$ to time step k (pulling from past to present)
- Tells you how the state would evolve in the absence of forcing functions
- Remember: this is still just a model. We don't really know what the state is, because the random forcing function messes things up!

Understanding Kalman Filter Notation
State Dynamics

- We are deliberately trying to keep the notation as simple as possible

- We could throw in other factors and terms.
  - A matrix Γ_k to distribute the Gaussian white noise to the states
  - An additional, deterministic control variable u_k, and associated distribution matrix Λ_k.

- For simplicity, we'll generally leave the control variable out of the picture.

The Kalman Filter Solution: State Extrapolation

- Given an estimate $\widehat{x}^{+}(k-1)$ at time $k-1$, we extrapolate it forward in time by the state transition matrix:

$$\widehat{x}^{-}(k) = \Phi_k \widehat{x}^{+}(k-1) \tag{4}$$

- Why is this the right extrapolation equation? Because the white noise forcing function is a zero mean process. Thus, on average, the equation is homogeneous and we should therefore use the homogeneous solution, i.e., the state transition matrix.

The Kalman Filter Solution: Covariance Extrapolation

- How certain are we that our solution is right?

- The answer is given by the **estimation error covariance matrix** or often just called the covariance matrix P_k.

- Just like the state estimate, it is propagated forward in time before being corrected:

  (5) $$P_k^- = \Phi_k P_{k-1}^+ \Phi_k^\top + \Gamma_k Q_k \Gamma_k^\top$$

  where Q_k is a matrix called the **process noise covariance matrix**

The Kalman Filter Solution: Covariance Extrapolation

- Generally, we'll assume that the noise distribution matrix Γ_k is the identity matrix. Thus, for covariance extrapolation we'll write instead of (5) the following:

$$P_k^- = \Phi_k P_{k-1}^+ \Phi_k^\top + Q_k \tag{6}$$

- Q_k measures the amount of dispersion in the white noise forcing function $w(k)$

- Later we'll learn how to define P_k.

The Kalman Filter Solution: Covariance Extrapolation

- What does Eq. (6) really mean?

- The term $\Phi_k P_{k-1}^{+} \Phi_k^{\top}$ represents the effect that the state dynamics has on the error covariance.

- The other term Q_k represents the increased uncertainty added into each step due to the process noise inherent in the system. This is due to the white noise in the system, possibly due to the aggregate effect of many unmodelled states.

The Kalman Filter Solution: The Kalman Gain

- The heart of the solution is the **Kalman Gain**: K_k

$$K_k = P_k^- H_k^\top \left[H_k P_k^- H_k^\top + R_k \right]^{-1} \tag{7}$$

where R_k is the measurement noise covariance matrix (to be defined later) governing the white measurement noise v_k in the measurement model:

$$z(k) = H_k x(k) + v(k)$$

The Kalman Filter Solution: State Update Equation

- Using the gain, the state update equation (which we've already seen) is

$$\widehat{x}^{+}(k) = \widehat{x}^{-}(k) + K_k \left[z(k) - H_k \widehat{x}^{-}(k) \right] \tag{8}$$

where z_k is the measurement and H_k is the measurment matrix at time k.

The Kalman Filter Solution: The Covariance Update Equation

- Likewise, the error covariance P_k is corrected using the gain:

$$P_k^+ = [I - K_k H_k] \, P_k^- \tag{9}$$

The Five Kalman Equations

(10) State Extrapolation: $\widehat{x}^{-}(k) = \Phi_k \widehat{x}^{+}(k-1)$

(11) Covariance Extrapolation: $P_k^{-} = \Phi_k P_{k-1}^{+} \Phi_k^{\top} + Q_k$

(12) Kalman Gain: $K_k = P_k^{-} H_k^{\top} \left[H_k P_k^{-} H_k^{\top} + R_k \right]^{-1}$

(13) State Update: $\widehat{x}^{+}(k) = \widehat{x}^{-}(k) + K_k \left[z(k) - H_k \widehat{x}^{-}(k) \right]$

(14) Covariance Update: $P_k^{+} = \left[I - K_k H_k \right] P_k^{-}$

Examples

- Scalar Example: Resistor Revisited
- Damped Harmonic Oscillator

Scalar Example: Resistor Revisited

- Number of states: 1
- System Dynamics: $x_k = x_{k-1}$
- Measurement Model: $z(k) = x(k) + v(k)$, where $v(k)$ is zero mean, Gaussian white noise with variance σ^2
- Thus, $\Phi_k = 1$, $H_k = 1$, $Q_k = 0$, $R_k = \sigma^2$

Scalar Example: Resistor Revisited

- State extrapolation and covariance extrapolation are simple:

$$\widehat{x}^{-}(k) = \widehat{x}^{+}(k-1)$$

$$P_k^{-} = P_{k-1}^{+}$$

  where P is a 1×1 matrix or scalar.

- The Kalman gain equation $K_k = P_k^{-} H_k^{\top} \left[H_k P_k^{-} H_k^{\top} + R_k \right]^{-1}$ becomes:

$$K_k = P_k^{-} \left(P_k^{-} + \sigma^2 \right)^{-1}$$

Scalar Example: Resistor Revisited

- State update equation $\widehat{x}^+(k) = \widehat{x}^-(k) + K_k \left[z(k) - H_k \widehat{x}^-(k) \right]$ becomes

$$\widehat{x}^+(k) = \widehat{x}^-(k) + \frac{P_k^-}{P_k^- + \sigma^2} \left(z(k) - \widehat{x}^-(k) \right)$$

- Covariance update equation $P_k^+ = [I - K_k H_k] P_k^-$ becomes

$$P_k^+ = \left(1 - \frac{P_k^-}{P_k^- + \sigma^2} \right) P_k^-$$

Damped Harmonic Oscillator

- Number of states = 2. System model:

$$\begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} = \exp\left(\begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \Delta t \right) \begin{bmatrix} x_1(k-1) \\ x_2(k-1) \end{bmatrix} + \begin{bmatrix} w_1(k) \\ w_2(k) \end{bmatrix}$$

- Thus, $\Phi_k = \exp\left(\begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \Delta t \right)$

- Here $\Delta t = t_k - t_{k-1}$

- Note that exp is a matrix exponential.

Damped Harmonic Oscillator

- Assume that the process noise w has the following statistics:

$$Q_k = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- This means that there is unity variance in position uncertainty and unity variance in velocity uncertainty.

- It also means that the uncertainties in position and velocity are uncorrelated (i.e., no non-zero off-diagonal elements in Q_k.

Dampled Harmonic Oscillator

- Measurement model:

$$z(k) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} + v(k)$$

  where the measurement noise $v(k)$ has variance σ^2

- Thus $H_k = \begin{bmatrix} 1 & 0 \end{bmatrix}$

- Note that we only have a position measurement.

Damped Harmonic Oscillator

- Φ_k can be approximated through series expansion of

$$\exp \begin{bmatrix} 0 & \Delta t \\ -k/m\Delta t & -c/m\Delta t \end{bmatrix}$$

- Or we can use Matlab's expm function for specific values of k,c,m, and Δt.

Damped Harmonic Oscillator

- Because Φ and Q are matrices, the state and covariance extrapolation equations do not simplify from their general matrix form.

- However, the gain equation $K_k = P_k^- H_k^\top \left[H_k P_k^- H_k^\top + R_k \right]^{-1}$ looks like this:

$$K_k = \begin{bmatrix} p_{11}^-(k) \\ p_{21}^-(k) \end{bmatrix} \left(\begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} p_{11}^-(k) \\ p_{21}^-(k) \end{bmatrix} + \sigma^2 \right)^{-1} = \begin{bmatrix} \dfrac{p_{11}^-(k)}{p_{11}^-(k) + \sigma^2} \\ \dfrac{p_{21}-(k)}{p_{11}^-(k) + \sigma^2} \end{bmatrix}$$

  where $p_{11}^-(k)$ and $p_{21}^-(k)$ are the (1,1) and (2,1) elements of P_k^-.

Damped Harmonic Oscillator

- State and covariance update equations left as an exercise!

Exercises 1. Crank through the resistor example using $\widehat{x}_0 = 100\Omega$, $\sigma = 0.1$, and $P_0 = 0$. Start the recursive loop with the gain equation first. Run through the loop three times.

2. Draw a rough plot of the error covariance, plotting both the extrapolated and updated covariance for any give k at the same time. (This will yield the classic Kalman sawtooth pattern.)

3. Similarly, draw a rough plot of the state estimates.

4. Write out the state update equation for the damped harmonic oscillator, using the Kalman gain derived in the example.

5. Write out the covariance update equation for the damped harmonic oscillator, using the Kalman gain dervied in the example.